# ¿Qué es Arduino?



### La placa (hardware)

El circuito físico con su chip, sus pines y su conector USB. La más común es la Arduino UNO.

### El IDE (software)

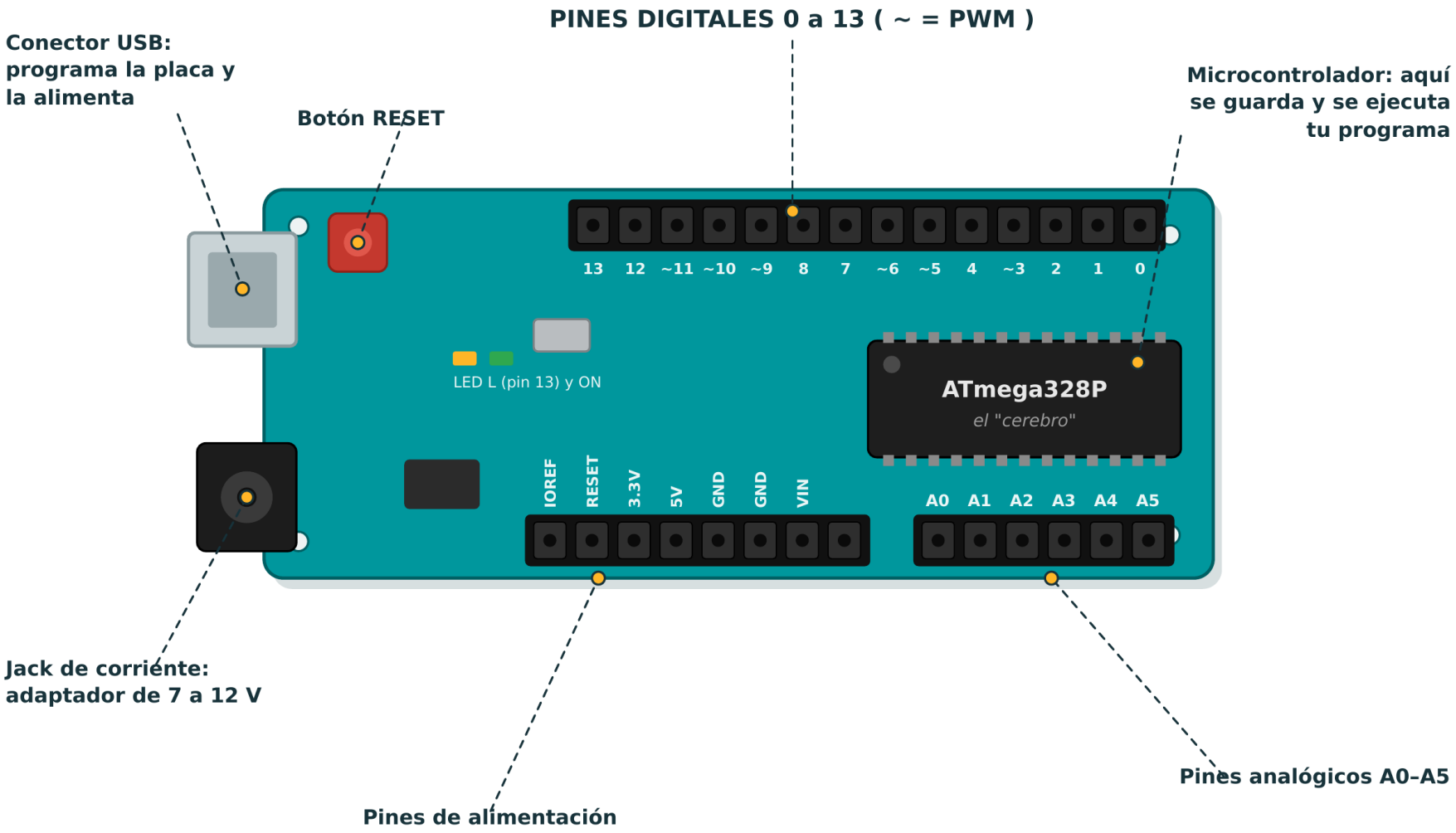
El programa gratuito donde escribes el código en el computador y lo envías a la placa por el cable USB.

### La comunidad

Miles de ejemplos, librerías y tutoriales gratis. Casi cualquier idea que se te ocurra ya la intentó alguien.

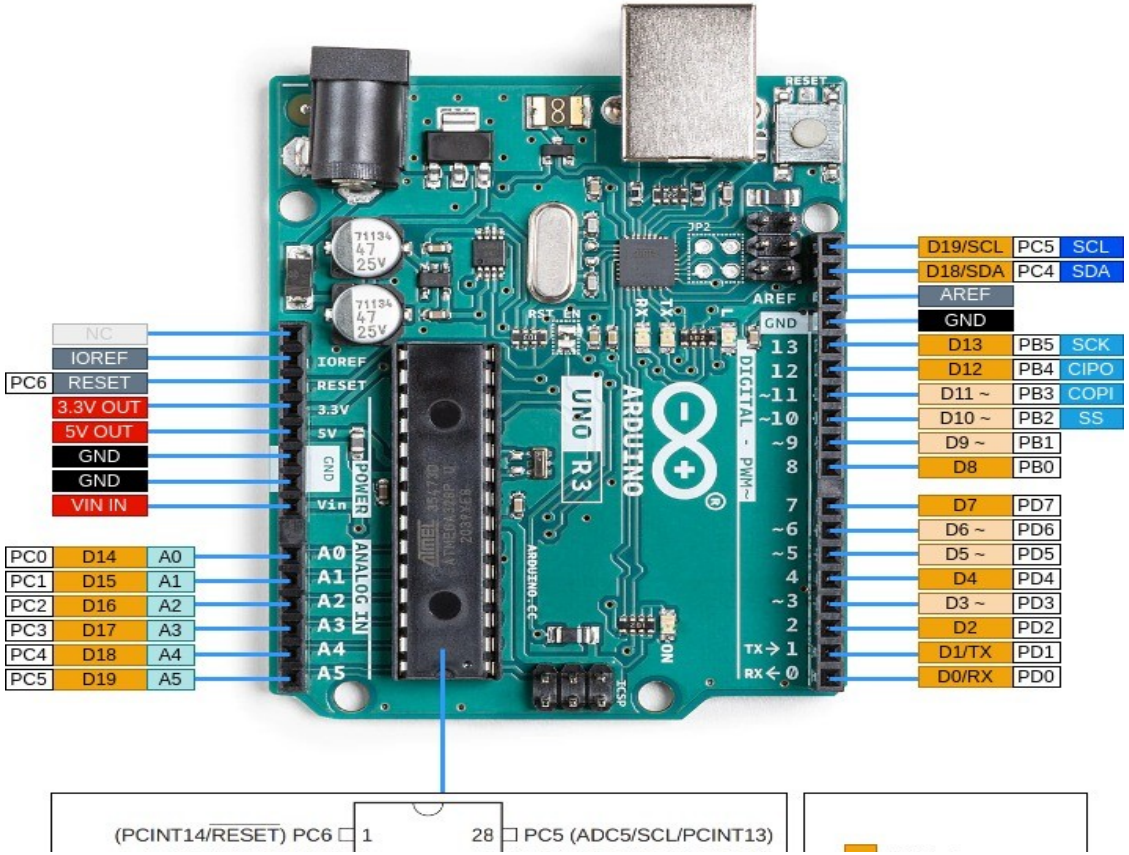
Una placa Arduino es una placa de desarrollo electrónico que integra un microcontrolador, junto con los componentes necesarios para alimentarlo, programarlo y conectarlo con sensores, actuadores y otros dispositivos electrónicos.

# Anatomía de una Arduino UNO



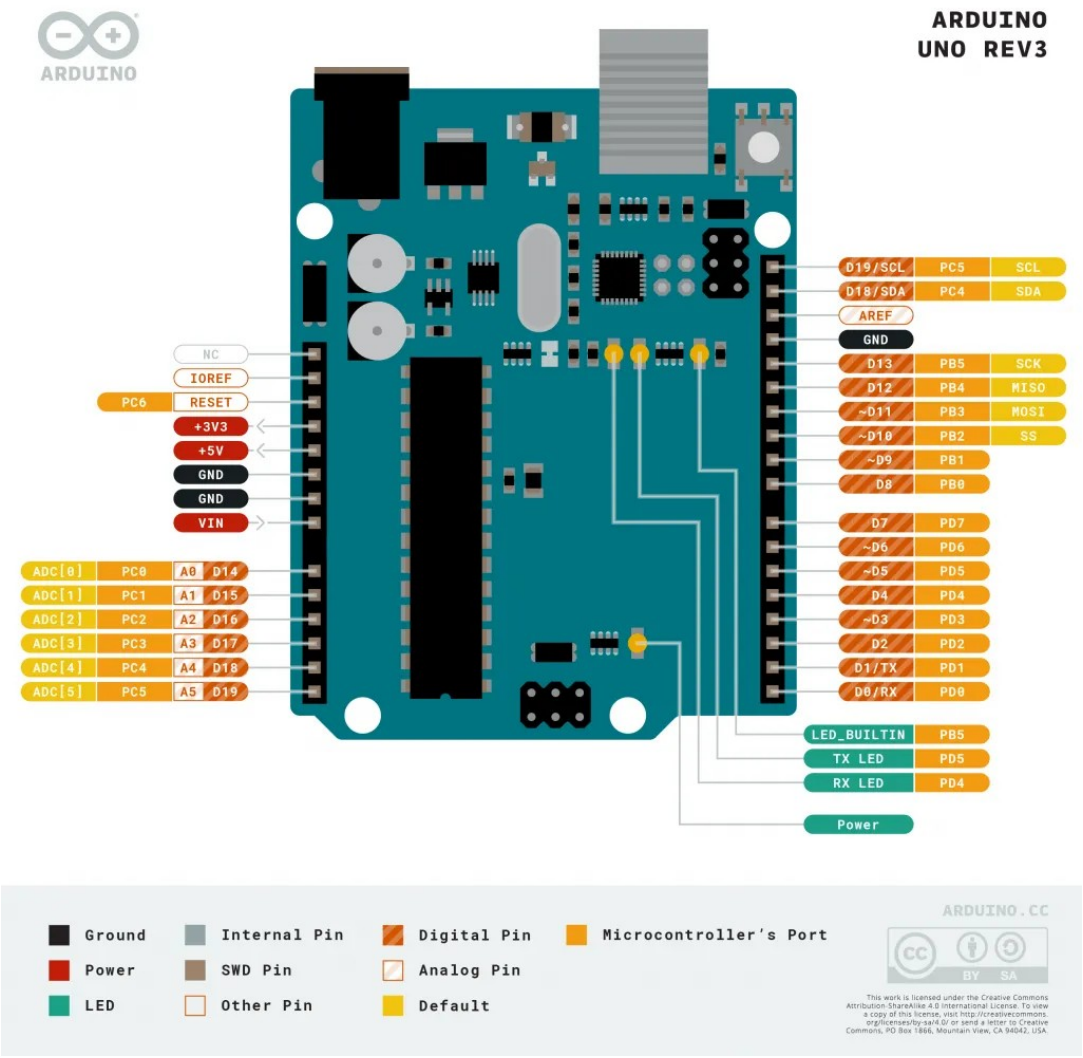

*Todas las placas Arduino comparten esta misma lógica: pines, alimentación y un microcontrolador.*

# Anatomía de una Arduino UNO




*Todas las placas Arduino comparten esta misma lógica: pines, alimentación y un microcontrolador.*

# Anatomía de una Arduino UNO




*Todas las placas Arduino comparten esta misma lógica: pines, alimentación y un microcontrolador.*

# Los pines: por dónde entra y sale la información

## PINES DIGITALES
**0 a 13**

Solo entienden 2 estados:

**HIGH = 5 V (encendido)**

**LOW = 0 V (apagado)**

Sirven para LEDs, pulsadores, zumbadores, relés…

Los que tienen ~ (3, 5, 6, 9, 10 y 11) simulan valores intermedios con PWM.

0 y 1 (RX/TX) se usan para comunicarse: mejor no usarlos.

## PINES ANALÓGICOS
**A0 a A5**

Leen un voltaje continuo entre 0 V y 5 V.

analogRead() devuelve un número entre 0 y 1023 (1024 escalones).

0 → 0 V
512 → 2,5 V
1023 → 5 V

Sirven para sensores de luz, temperatura, potenciómetros…

## PINES DE ENERGÍA
**5V · 3.3V · GND · VIN**

**5V entrega 5 voltios**

**3.3V entrega 3,3 voltios**

**GND el polo negativo (0 V)**

**VIN entrada de 7–12 V**

TODO circuito debe volver a un pin GND: sin GND no se cierra el camino y nada funciona.

Hay 3 pines GND y todos son equivalentes.

# ¿Cómo se alimenta el Arduino?



## 1 Cable USB

**5 V desde el computador o un cargador de celular**

La más común, además sirve para subir el programa.



**Corriente: ≈ 500 mA**

## 2 Jack de corriente

**Adaptador de pared de 7 V a 12 V**

El regulador de la placa baja el voltaje a 5 V. Ideal si el proyecto queda fijo.



**Corriente: hasta 1 A**

## 3 Pin VIN + GND

**Batería de 9 V o porta-pilas (6–12 V)**

Para proyectos que se mueven, se debe tener cuidado con la polaridad.

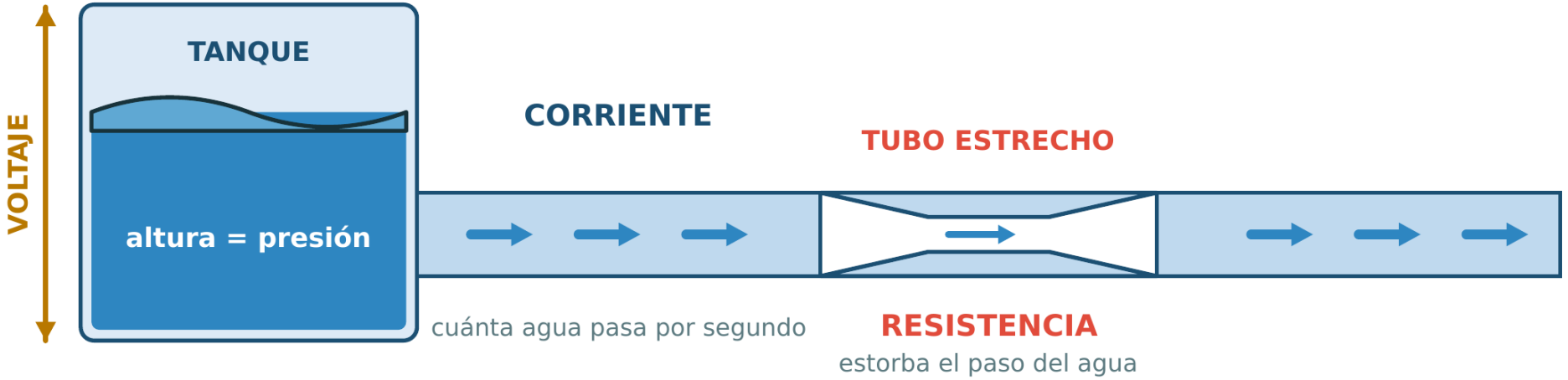



**Corriente: según batería**

⚠ **Nunca conectes el USB y una batería con la polaridad invertida, y nunca apliques más de 12 V.**
Cada pin digital entrega como máximo 40 mA (lo seguro son 20 mA) y toda la placa no debe pasar de 200 mA.

# Voltaje, corriente y resistencia

**La electricidad se entiende fácil si la imaginas como agua en una tubería**

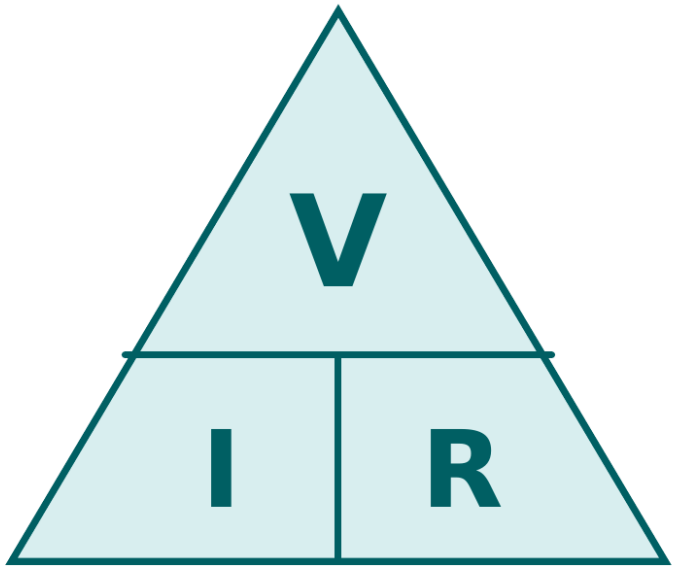



| | |
|---|---|
| **VOLTAJE (V)**<br>se mide en Voltio | La fuerza que empuja a los electrones.<br>*Como la presión del agua.* |
| **CORRIENTE (I)**<br>se mide en Amperio | La cantidad de electrones que pasan.<br>*Como el caudal del agua.* |
| **RESISTENCIA (R)**<br>se mide en Ohmio Ω | Lo que se opone al paso de la corriente.<br>*Como un tubo más angosto.* |

# La ley de Ohm



Relaciona las tres magnitudes: si conoces dos, calculas la tercera




*Tapa la que buscas y lee la fórmula*

**V = I × R**
Voltaje = corriente × resistencia

**I = V ÷ R**
Corriente = voltaje ÷ resistencia

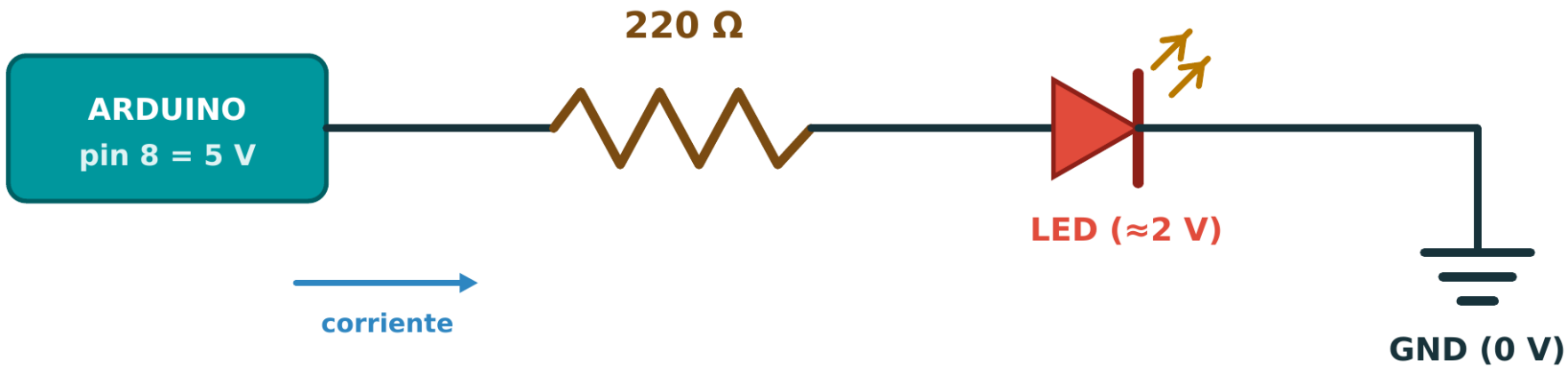
**R = V ÷ I**
Resistencia = voltaje ÷ corriente

**Ejemplo:** **un pin de 5 V con una resistencia de 220 Ω → I = 5 V ÷ 220 Ω = 0,0227 A = 22,7 mA**

Recuerda: 1 A = 1000 mA. Los LEDs trabajan cómodos entre 10 mA y 20 mA.

# Por qué el LED siempre lleva resistencia

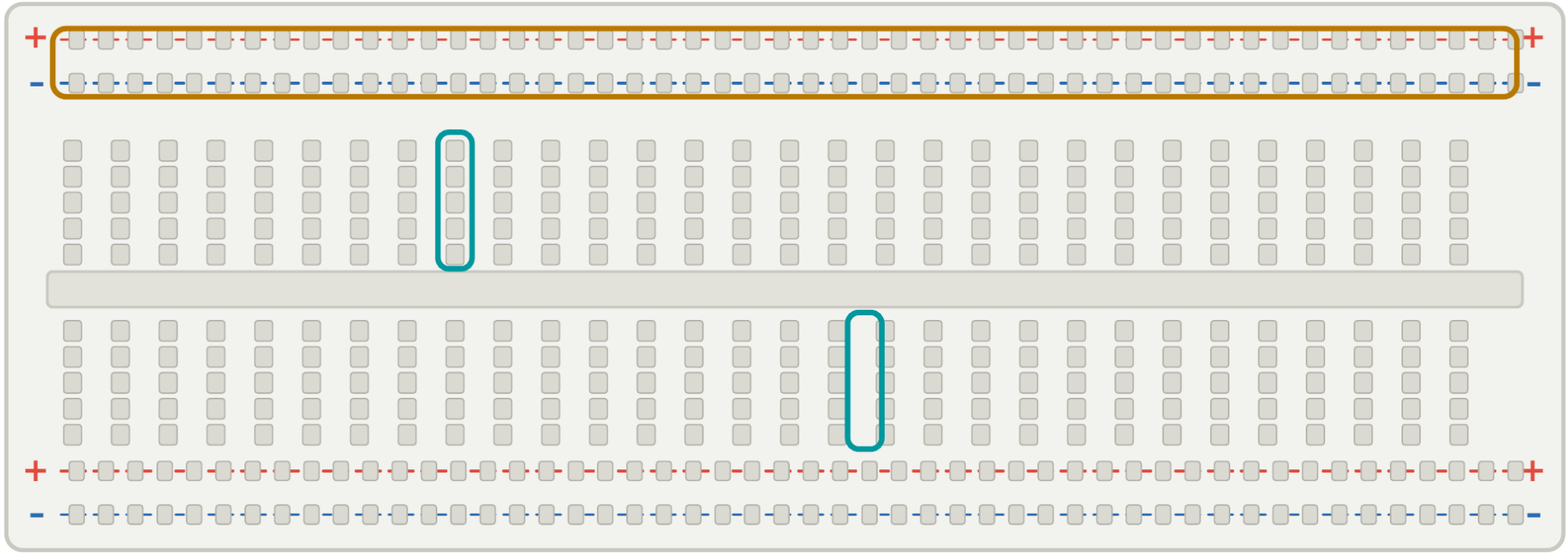
Un LED conectado directo a 5 V recibe demasiada corriente y se quema (y puede dañar el pin)




## El cálculo, paso a paso

1. El pin entrega 5 V y el LED “gasta” unos 2 V → sobran $5 - 2 = 3$ V para la resistencia.
2. Queremos una corriente segura de 20 mA, es decir 0,02 A.
3. Aplicamos la ley de Ohm: $R = V \div I = 3\ V \div 0{,}02\ A = 150\ \Omega$
4. **En el mercado la más parecida (y más segura) es la de 220 Ω → esa usamos.**

# La protoboard: conectar sin soldar



- **Los 5 huecos de una misma columna están unidos por dentro.**
  Si metes una pata del LED y un cable en la misma columna, quedan conectados.
- **Las 2 filas largas de arriba y de abajo (+ y −) recorren toda la placa.**
  Se usan para repartir los 5 V y el GND del Arduino a todo el circuito.
- **El canal del centro separa la mitad de arriba de la de abajo.**
  Por eso los chips se montan “a caballo” sobre el canal.

# El Arduino IDE por dentro

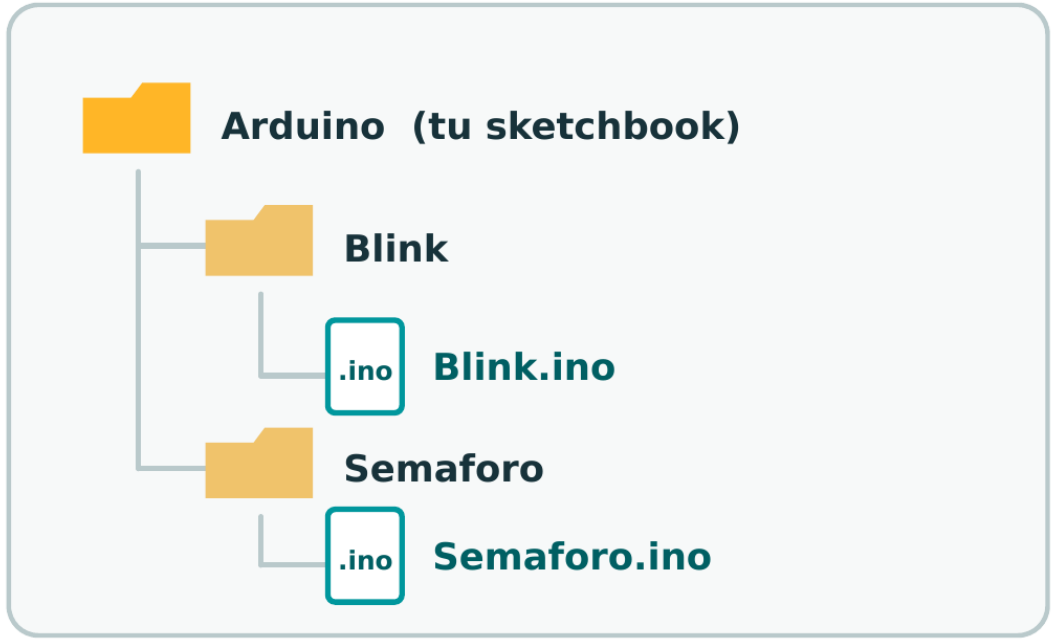





1. Verificar: revisa que no haya errores y traduce el código.
2. Subir: envía el programa a la placa por el cable USB.
3. Placa y puerto: elige “Arduino Uno” y el puerto COM correcto.
4. Editor: aquí escribes tu sketch.
5. Consola: mensajes de error o de éxito. ¡Léela siempre!

# Cómo se guarda un programa






| | |
|---|---|
| **Ctrl + S** | guarda el sketch en el computador. |
| **.ino** | es la extensión de los archivos de Arduino. |
| **Una carpeta** | por sketch, con el mismo nombre. |
| **Sin tildes** | ni espacios ni ñ: usa Semaforo_v2. |
| **Ejemplos** | el menú Archivo ▸ Ejemplos trae muchos. |

**Ojo con esto**

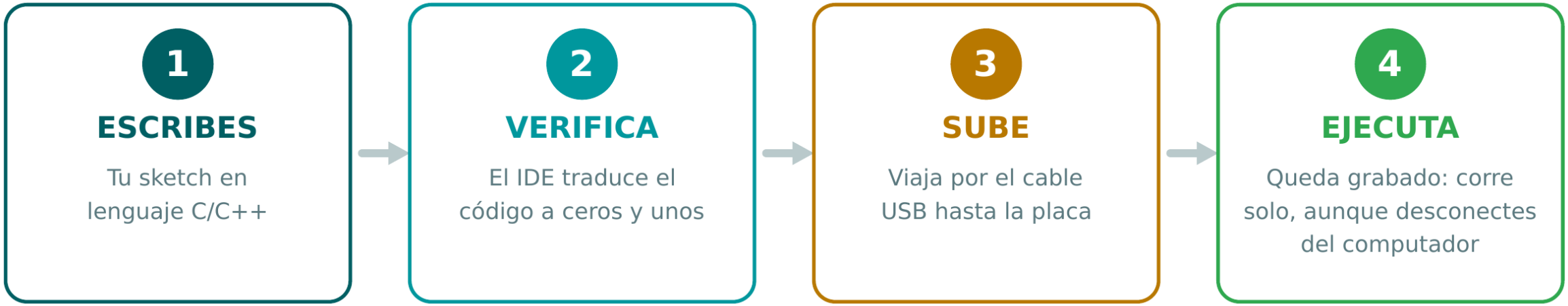
Guardar el sketch en el computador NO lo sube a la placa: son dos cosas distintas.

El código dentro del Arduino solo cambia cuando presionas el botón Subir (→).

# Del código a la placa



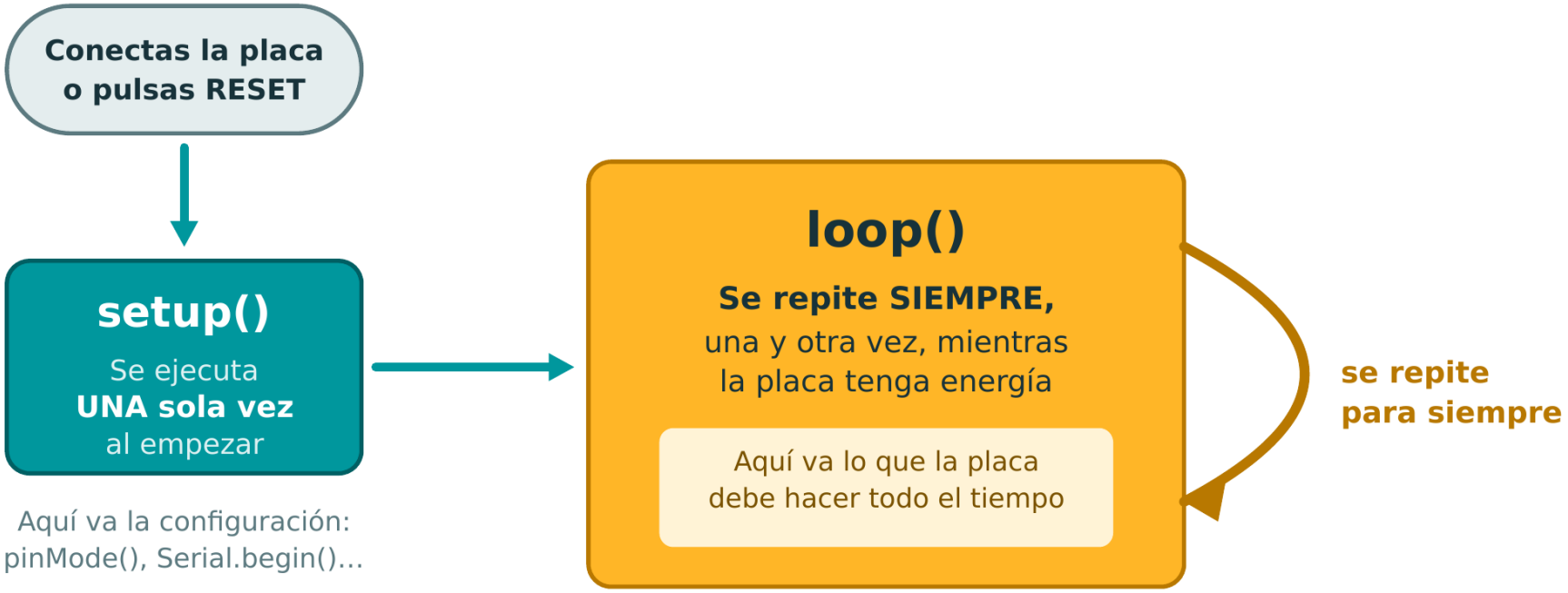
## ¿Qué pasa cuando presionas “Subir”?




**El programa se guarda en la memoria FLASH del microcontrolador: no se borra al apagar.**

# La estructura de todo sketch: setup() y loop()

## Todo sketch tiene dos bloques obligatorios




**Importante:** el Arduino NO tiene botón de “ejecutar”. Al recibir energía corre setup() y luego loop() sin parar.

Si el programa se queda “pegado” dentro de un delay() muy largo, la placa no puede hacer nada más en ese rato.

# Un sketch comentado línea por línea



estructura.ino

```
1  // 1) Zona de declaraciones: variables y constantes globales
2  const int LED = 13;
3
4  // 2) setup(): se ejecuta UNA sola vez al encender o al pulsar RESET
5  void setup() {
6    pinMode(LED, OUTPUT);   // configura el pin 13 como salida
7  }
8
9  // 3) loop(): se repite INFINITAMENTE mientras haya energía
10 void loop() {
11   digitalWrite(LED, HIGH);  // enciende
12   delay(1000);              // espera 1000 ms = 1 segundo
13   digitalWrite(LED, LOW);   // apaga
14   delay(1000);
15 }
```

*Todo lo que va después de // es un comentario: el Arduino lo ignora, pero tu profesor y tu yo del futuro lo agradecen.*

# Variables: tipos de datos

| TIPO | PARA QUÉ SIRVE | QUÉ VALORES ACEPTA | EJEMPLO |
|---|---|---|---|
| `int` | Números enteros | −32.768 a 32.767 | `int contador = 0;` |
| `long` | Enteros muy grandes | ±2.147.483.647 | `long tiempo = 250000;` |
| `float` | Números con decimales | $\pm 3{,}4 \times 10^{38}$ | `float temp = 24.5;` |
| `bool` | Verdadero o falso | true / false | `bool activo = true;` |
| `char` | Un solo carácter | 'A' ... 'z' | `char letra = 'A';` |
| `byte` | Enteros pequeños | 0 a 255 | `byte nivel = 200;` |
| `String` | Texto (cadena) | "Hola Arduino" | `String msg = "Hola";` |

**Regla práctica: usa int para casi todo; float solo cuando de verdad necesites decimales.**

# Declarar variables y constantes



variables.ino

```
// TIPO  nombre  =  valor inicial;
int    contador    = 0;          // entero:  -32768 a 32767
long   microsegundos = 250000; // entero grande
float  temperatura = 24.5;     // decimales
bool   encendido    = true;     // true / false
char   letra        = 'A';      // un solo carácter
byte   nivel        = 200;      // 0 a 255 (1 byte)
String mensaje      = "Hola Arduino";

// CONSTANTE: su valor no puede cambiar durante el programa
const int PIN_LED = 9;

void setup() {
  contador = contador + 1;    // las variables SÍ cambian
  temperatura += 0.5;         // forma corta de sumar
  // PIN_LED = 10;  <-- ERROR: una constante no se puede modificar
}
```

*Estructura: tipo + nombre + = + valor + ;  ·  const significa que el valor nunca cambia.*

# Decisiones y ciclos: if, for y while



control.ino

```
int valor = 7;

void setup() {
  // ---- Condicional: decide entre caminos ----
  if (valor > 10) {
    digitalWrite(13, HIGH);
  } else if (valor == 7) {
    digitalWrite(12, HIGH);
  } else {
    digitalWrite(11, HIGH);
  }

  // ---- Ciclo for: repite un número conocido de veces ----
  for (int i = 2; i <= 7; i++) {
    pinMode(i, OUTPUT);      // configura los pines 2, 3, 4, 5, 6 y 7
  }

  // ---- Ciclo while: repite MIENTRAS se cumpla la condición ----
  while (valor > 0) {
    valor--;                 // resta 1 (si no, sería un ciclo infinito)
  }
}
```

*Con estas tres estructuras se puede programar prácticamente cualquier cosa.*

# Las funciones que más vas a usar

### pinMode(pin, modo)

Define si un pin es entrada o salida.

```
pinMode(8, OUTPUT);
```

### digitalWrite(pin, valor)

Pone un pin en HIGH (5 V) o LOW (0 V).

```
digitalWrite(8, HIGH);
```

### digitalRead(pin)

Lee si un pin está en HIGH o en LOW.

```
estado = digitalRead(2);
```

### delay(milisegundos)

Pausa el programa. 1000 ms = 1 segundo.

```
delay(500);
```

### analogRead(pin)

Lee un sensor: devuelve de 0 a 1023.

```
luz = analogRead(A0);
```

### analogWrite(pin, valor)

PWM: brillo o velocidad, de 0 a 255.

```
analogWrite(9, 128);
```

**Todas van dentro de setup() o de loop(), y siempre terminan en punto y coma ;**

# Encender un LED: el sketch “Blink”



Blink.ino

```
/*
  Blink - Enciende y apaga un LED cada segundo.
  Circuito: pin 8 -> resistencia 220 Ω -> LED -> GND
*/

void setup() {
  pinMode(8, OUTPUT);          // el pin 8 será una SALIDA
}

void loop() {
  digitalWrite(8, HIGH);       // HIGH = 5 V  -> el LED enciende
  delay(1000);                 // mantener 1 segundo
  digitalWrite(8, LOW);        // LOW  = 0 V  -> el LED apaga
  delay(1000);                 // mantener 1 segundo
}
```

*Montaje: pin 8 → resistencia de 220 Ω → pata larga del LED · pata corta del LED → GND.*

EJEMPLO 2

# El mismo LED, pero con nombres claros



BlinkMejorado.ino

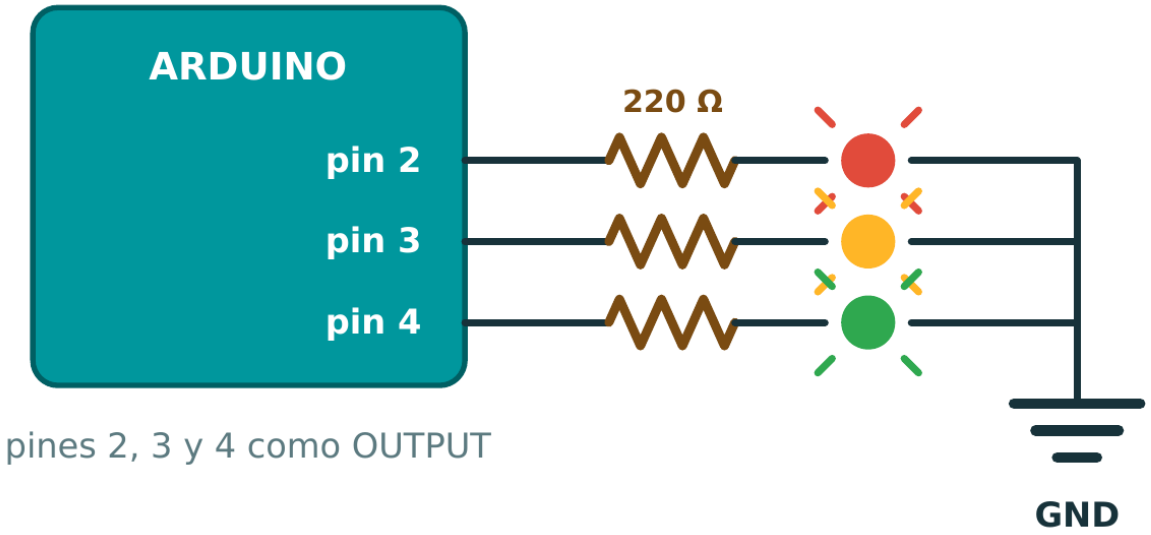
```ino
// Misma idea, pero con nombres claros: más fácil de leer y modificar
const int LED_ROJO   = 8;    // si cambio de pin, solo edito esta línea
const int ESPERA_MS  = 250;  // velocidad del parpadeo

void setup() {
  pinMode(LED_ROJO, OUTPUT);
}

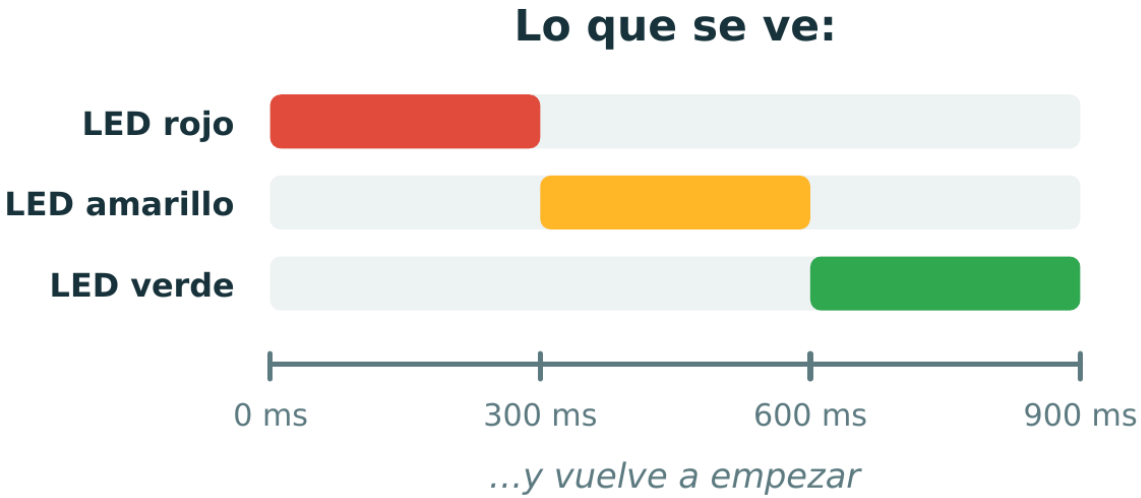
void loop() {
  digitalWrite(LED_ROJO, HIGH);
  delay(ESPERA_MS);
  digitalWrite(LED_ROJO, LOW);
  delay(ESPERA_MS);
}
```

*Usar constantes al inicio hace que el código se entienda y que cambiar de pin sea cuestión de una línea.*

# Secuencia de tres LEDs: el montaje




pines 2, 3 y 4 como OUTPUT




**La receta es siempre la misma:**
encender → esperar → apagar → pasar al siguiente.

**delay(300) marca el ritmo.**
Si bajas el número, la secuencia va más rápido.

**¿Muchos LEDs?**
Usa un ciclo for en vez de repetir líneas: menos código y menos errores.

# Secuencia de tres LEDs: el código



Secuencia3LED.ino

```
// Enciende tres LEDs uno detrás de otro
const int ROJO = 2, AMARILLO = 3, VERDE = 4;
const int T = 300;                          // duración de cada paso (ms)

void setup() {
  pinMode(ROJO, OUTPUT);
  pinMode(AMARILLO, OUTPUT);
  pinMode(VERDE, OUTPUT);
}

void loop() {
  digitalWrite(ROJO, HIGH);  delay(T);  digitalWrite(ROJO, LOW);
  digitalWrite(AMARILLO, HIGH); delay(T); digitalWrite(AMARILLO, LOW);
  digitalWrite(VERDE, HIGH); delay(T);  digitalWrite(VERDE, LOW);
}
```

*Fíjate en el patrón: encender → esperar → apagar. Se repite para cada LED.*

# Barrido tipo “auto fantástico” con un ciclo for



BarridoLEDs.ino

```
// 6 LEDs en los pines 2 a 7: efecto "auto fantástico"
const int PRIMERO = 2;
const int ULTIMO  = 7;
const int T = 120;

void setup() {
  for (int pin = PRIMERO; pin <= ULTIMO; pin++) {
    pinMode(pin, OUTPUT);           // configura los 6 pines de una vez
  }
}

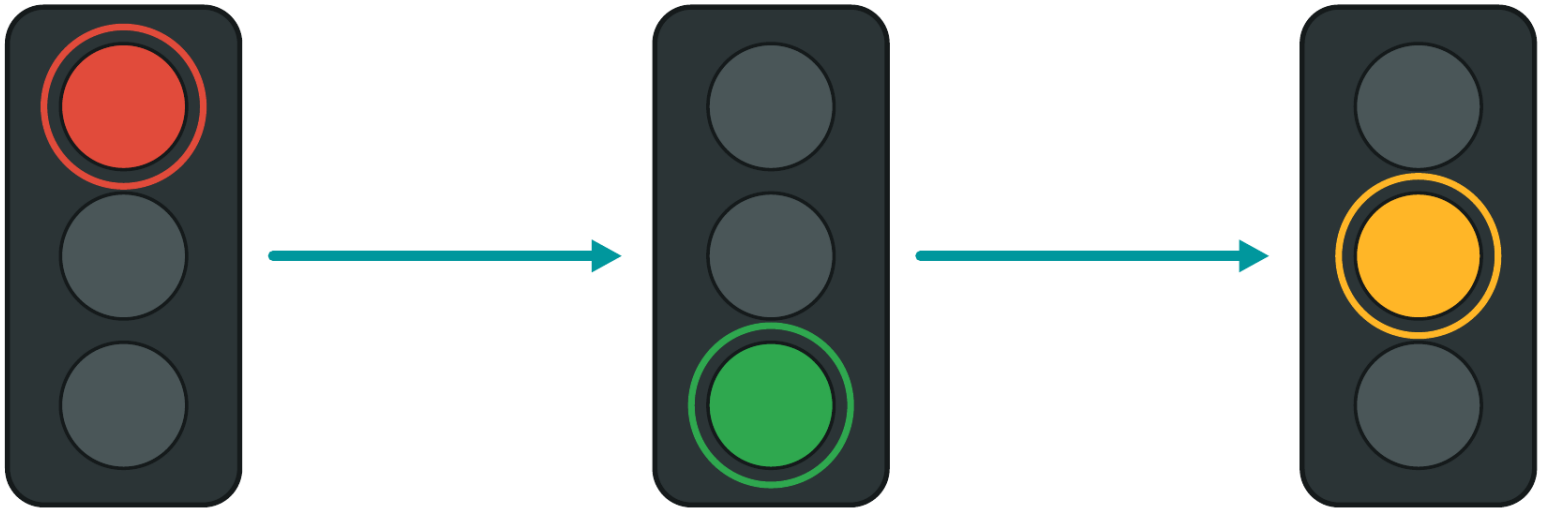
void loop() {
  for (int pin = PRIMERO; pin <= ULTIMO; pin++) {   // ida  ->
    digitalWrite(pin, HIGH); delay(T); digitalWrite(pin, LOW);
  }
  for (int pin = ULTIMO; pin >= PRIMERO; pin--) {   // vuelta <-
    digitalWrite(pin, HIGH); delay(T); digitalWrite(pin, LOW);
  }
}
```

*Seis LEDs en los pines 2 a 7 y solo dos ciclos for: ida y vuelta.*

# Un semáforo de verdad



Tres estados que se repiten en orden, cada uno con su propia duración



**ROJO**
**durante 5 s**
Detenerse

**VERDE**
**durante 4 s**
Avanzar

**AMARILLO**
**durante 1,5 s**
Precaución

**y vuelve a empezar, para siempre**

# El código del semáforo



Semaforo.ino

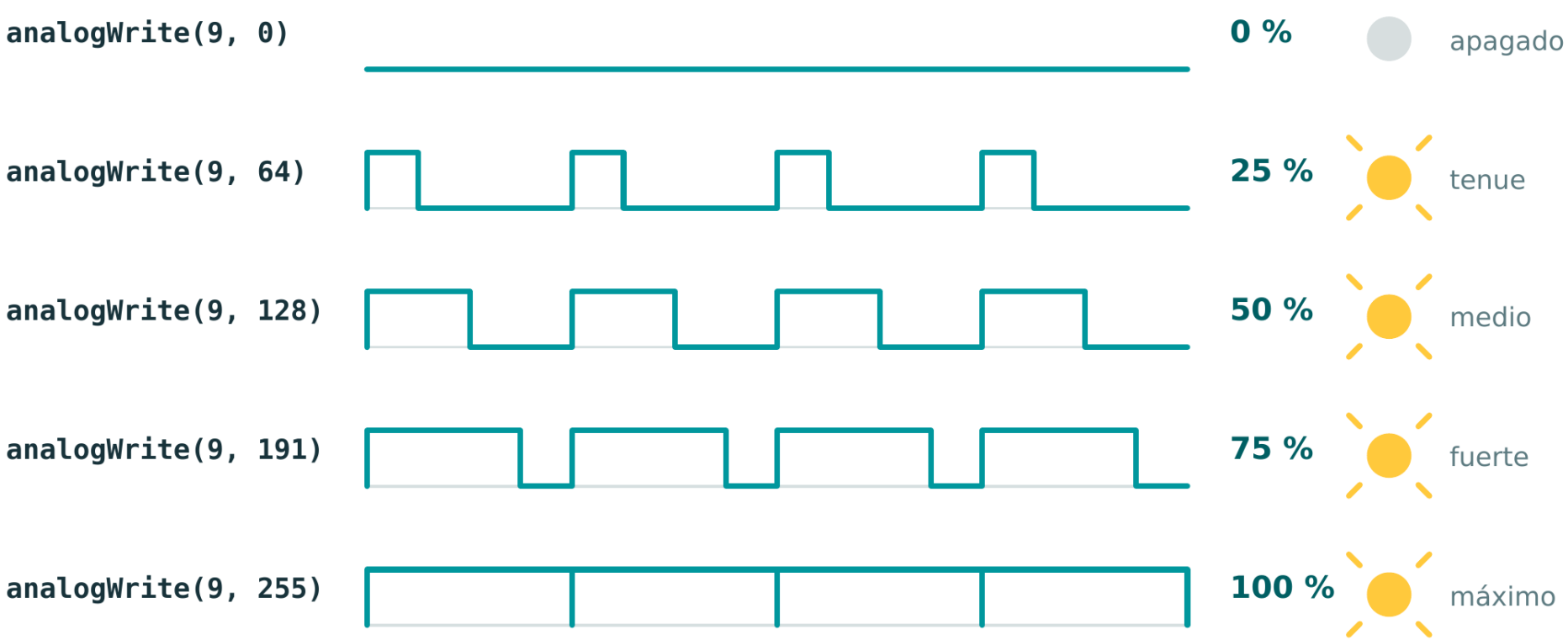
```
1  const int ROJO = 2, AMARILLO = 3, VERDE = 4;
2
3  void setup() {
4    pinMode(ROJO, OUTPUT);
5    pinMode(AMARILLO, OUTPUT);
6    pinMode(VERDE, OUTPUT);
7  }
8
9  void loop() {
10   luz(ROJO, 5000);         // 5 s en rojo
11   luz(VERDE, 4000);        // 4 s en verde
12   luz(AMARILLO, 1500);     // 1.5 s en amarillo
13 }
14
15 // Función propia: enciende SOLO un color durante 'tiempo' milisegundos
16 void luz(int color, int tiempo) {
17   digitalWrite(ROJO, LOW);
18   digitalWrite(AMARILLO, LOW);
19   digitalWrite(VERDE, LOW);
20   digitalWrite(color, HIGH);
21   delay(tiempo);
22 }
```

*Aquí creamos nuestra propia función, luz(), para no repetir las mismas cuatro líneas tres veces.*

# PWM: brillos intermedios



El pin se enciende y apaga miles de veces por segundo. El ojo ve el promedio.




**Solo funciona en los pines marcados con ~ : 3, 5, 6, 9, 10 y 11.  Los valores van de 0 a 255.**

# El código del efecto “fade”



Fade.ino

```
// PWM: el LED sube y baja de brillo suavemente
const int LED = 9;           // debe ser un pin con el símbolo ~

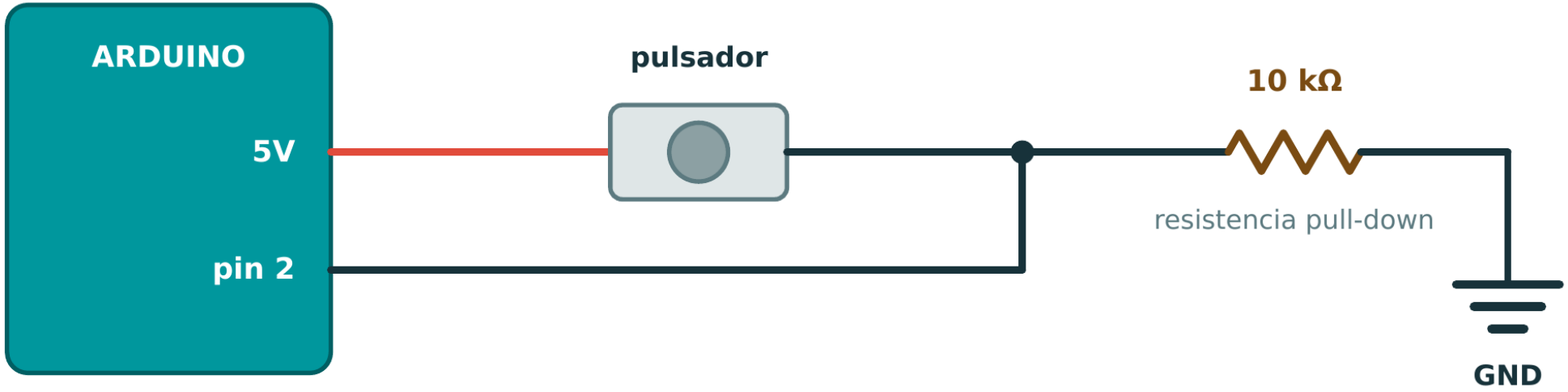
void setup() {
  pinMode(LED, OUTPUT);
}

void loop() {
  for (int brillo = 0; brillo <= 255; brillo++) {   // sube el brillo
    analogWrite(LED, brillo);   // 0 = apagado ... 255 = brillo máximo
    delay(8);
  }
  for (int brillo = 255; brillo >= 0; brillo--) {   // baja el brillo
    analogWrite(LED, brillo);
    delay(8);
  }
}
```

*Recuerda: analogWrite() solo funciona en los pines marcados con ~.*

# Leer un pulsador: la primera entrada






- **Botón presionado:** los 5 V llegan al pin 2 → digitalRead() devuelve HIGH (1)
- **Botón suelto:** la resistencia lleva el pin a GND → digitalRead() devuelve LOW (0)
- **¿Y si no pongo la resistencia?:** el pin queda “flotando” y lee valores al azar: el LED parpadea solo
- **Atajo profesional:** pinMode(2, INPUT_PULLUP) usa la resistencia interna del chip

# El código del pulsador



Pulsador.ino

```
const int BOTON = 2;          // entrada
const int LED   = 8;          // salida
int estado = 0;               // guarda la lectura del pulsador

void setup() {
  pinMode(BOTON, INPUT);      // con pull-down de 10 kΩ a GND
  pinMode(LED, OUTPUT);
}

void loop() {
  estado = digitalRead(BOTON);   // lee: HIGH (1) o LOW (0)

  if (estado == HIGH) {          // si el botón está presionado...
    digitalWrite(LED, HIGH);
  } else {
    digitalWrite(LED, LOW);
  }
}
```

*digitalRead() devuelve HIGH o LOW; el if decide qué hacer con esa respuesta.*

# El Monitor Serie: depurar y ver datos






### ¿Para qué sirve?

Para ver qué está “pensando” la placa: valores de sensores, mensajes, errores.

### Serial.begin(9600);

va en setup(). Abre la comunicación a 9600 baudios.

### Serial.print(x);

escribe sin bajar de línea.

### Serial.println(x);

escribe y baja de línea.

### Regla de oro

la velocidad del monitor (abajo a la derecha) debe coincidir con la del código.

# Errores comunes (y cómo salir de ellos)

### Falta el punto y coma ;

El IDE señala la línea siguiente, no la del error. Revisa la de arriba.

### LED sin resistencia

Se quema el LED y puedes dañar el pin. Siempre 220 Ω.

### Olvidar el GND

Sin retorno a GND el circuito no se cierra: nada enciende.

### LED al revés

La pata larga (+) va al pin; la corta (−) hacia GND.

### Puerto o placa mal elegidos

Si dice “no se encontró la placa”, revisa Herramientas ▸ Puerto.

### Mayúsculas equivocadas

digitalwrite ≠ digitalWrite. C++ distingue mayúsculas.

### delay() gigante

Con delay(10000) la placa queda “congelada” 10 segundos.

### Cambiar el código y no subirlo

Guardar no basta: hay que presionar Subir (→).

CONCEPTO CLAVE

# Los tres tipos de pines del Arduino UNO

| | **Digitales**<br>pines 0 – 13 | **Digitales PWM ~**<br>pines 3, 5, 6, 9, 10, 11 | **Analógicos**<br>pines A0 – A5 |
|---|---|---|---|
| **¿Qué leen?** | Solo 2 estados:<br>**LOW** (0 V) o **HIGH** (5 V) | Igual que los digitales:<br>**LOW** o **HIGH** | **Cualquier voltaje** de 0 a 5 V,<br>como un número de **0 a 1023** |
| **¿Qué escriben?** | Solo **0 V** o **5 V** | 0 V, 5 V **o un “promedio”**<br>con valores de **0 a 255** | Solo 0 V o 5 V (como digitales).<br>**No pueden hacer PWM** |
| **Leer con…** | `digitalRead()` | `digitalRead()` | `analogRead()` y `digitalRead()` |
| **Escribir con…** | `digitalWrite()` | `digitalWrite()` y `analogWrite()` | `digitalWrite()` |
| **Sensores** | Digitales: pulsador,<br>sensor de movimiento | Digitales | Analógicos (potenciómetro, luz,<br>temperatura) **y** digitales |
| **Es como…** | 💡 un **interruptor**:<br>prendido o apagado | ⚡ un interruptor **tan rápido**<br>que parece un regulador | 🌡️ un **termómetro**:<br>mide “cuánto” |

⚠️ **analogWrite() funciona en los pines con ~ · ¡NO en A0 – A5! A0 – A5 sirven para LEER.**

RESUMEN

# ¿Qué pin uso?

| | **DIGITAL**<br>solo 0 V o 5 V | **ANALÓGICO**<br>valores intermedios |
|---|---|---|
| ✏️ **ESCRIBIR**<br>el Arduino habla | **Todos los pines**<br>`digitalWrite(pin, HIGH)` | **Solo los PWM ~**<br>`~3` `~5` `~6` `~9` `~10` `~11`<br>`analogWrite(pin, 0 … 255)` |
| 👂 **LEER**<br>el Arduino escucha | **Todos los pines**<br>`digitalRead(pin)` | **Solo los analógicos**<br>`A0` `A1` `A2` `A3` `A4` `A5`<br>`analogRead(pin) → 0 … 1023` |

⚠️ **Pines 0 y 1: mejor no usarlos. El Arduino los necesita para hablar con el computador.**